

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа 4

Выполнил:

Филатов Арсений Сергеевич

Группа К3339

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

1. Цель работы

Сформировать технический документ по разделению монолитного backend-приложения «платформа поиска работы» на микросервисы с соблюдением концепции отдельной базы данных на каждый сервис (database-per-service), описать связи между сервисами, способы их взаимодействия, межсервисные контракты в формате OpenAPI и план миграции с монолита. Результат служит основой для практической реализации в ЛР2.

2. Задачи

- Разделить монолитное приложение на микросервисы в логике database-per-service.
- Спроектировать архитектуру: связи сервисов, каналы взаимодействия (HTTP REST для межсервисного обмена, единый API-шлюз для клиента).
- Спроектировать разбиение одной БД монолита на изолированные БД по сервисам.
- Спроектировать и задокументировать эндпоинты межсервисного взаимодействия в формате спецификации OpenAPI 3.x.
- Описать типовые запросы и ответы, задокументировать коды и формат ошибок.
- Сформулировать конкретные требования и пошаговый план достижения разделения..

3. Исходный монолит (ЛР1, каталог lab1)

Стек: Bun, Elysia, Prisma, PostgreSQL. Публичный префикс API: /api/v1.

Предметная область: пользователи и роли, профиль соискателя, резюме (образование, опыт, навыки M:N), компании и членства работодателя, вакансии (статусы, отрасль, уровень опыта, навыки), отклики с уникальностью (vacancyId, userId). Справочники: отрасли, уровни опыта, навыки. При проектировании микросервисов сохраняется семантика API для клиента; меняется внутренняя декомпозиция и физическое хранение данных.

4. Целевая архитектура и разбиение БД

Выделенные сервисы:

1. auth учётные записи: роли, пользователи, refresh-токены
2. catalog справочники: отрасли, уровни опыта, навыки
3. jobseeker профиль соискателя, резюме, блоки образования и опыта, связь резюме с skillId (логическая связь с каталогом)
4. employer компании, членства работодателя, вакансии, навыки вакансии.
5. application отклики (связь vacancyId, userId, resumeId без FK между БД).
6. gateway — единая точка входа для клиента (маршрутизация по пути запроса)

Между базами не используются внешние ключи: идентификаторы согласованы на уровне доменной модели и проверок в коде сервисов. Создание отдельных БД в одном экземпляре PostgreSQL описано в скрипте docker/init-databases.sql (GRANT на пользователя приложения).

5. Взаимодействие сервисов

- Клиент обращается только к шлюзу (например, порт 3000); пути /api/v1/* проксируются на соответствующий сервис.
- Пользовательская аутентификация: JWT access (с claim role и sub=userId), общий секрет JWT_SECRET; refresh хранится в БД auth.

- Межсервисные вызовы: HTTP, заголовок x-internal-secret (INTERNAL_SECRET).

Примеры:

- catalog: GET /internal/industries/:id, GET /internal/experience-levels/:id, POST /internal/skills/validate, GET /internal/skills?ids=...
- jobseeker: GET /internal/resumes/:resumeId?ownerUserId=...
- employer: GET /internal/vacancies/:vacancyId, GET /internal/vacancies/:vacancyId/employer-access?userId=...

Сервисы catalog, jobseeker, employer, application используют внутренние вызовы для валидации ссылок и прав без дублирования справочных данных в чужих БД.

7. Результаты

Подготовлен целостный технический дизайн: состав сервисов, изоляция БД, диаграмма зависимостей, описание internal OpenAPI, единый формат ошибок и план миграции.

8. Выводы

Концепция database-per-service позволяет независимо развивать границы данных и команды, но требует явных контрактов между сервисами и отказа от транзакций через бывшие FK. HTTP и OpenAPI для internal API дают проверяемое описание зависимостей; шлюз сохраняет привычный контракт для клиентов